

UNIVERSITY OF NOVI SAD
FACULTY OF SCIENCES
DEPARTMENT OF MATHEMATICS AND
INFORMATICS



Minimax algorithm on the game of SIM

Artificial intelligence

-Seminar paper-

Author:
Mia Temerinac

Novi Sad, 2026.

Contents

Abstract **2**

1 Game of SIM **3**
1.1 The setting of the game . . . 3
1.2 A winning strategy for the second player . . . 3

2 Minimax algorithm in SIM **4**
2.1 Definiton of Minimax . . . 4
2.1.1 Function getMax . . . 4
2.1.2 Function getMin . . . 5
2.1.3 Implementation Details . . . 5
2.2 Properties of the Algorithm . . . 6
2.2.1 Time complexity . . . 6
2.2.2 Space complexity . . . 7

3 Alpha-Beta pruning **7**
3.1 Definition and implementation details . . . 7
3.2 Time complexity . . . 8

4 Depth-limited search using Heuristic evaluation **9**
4.1 Heuristic evaluation function . . . 9
4.2 Properties of the algorithm . . . 10

5 Comparison analysis **10**
5.1 Time and space complexity comparison . . . 10
5.2 Comparison across different graph sizes . . . 11

Conclusion **13**

References **13**

Abstract

The study of various deterministic, zero-sum games and the simulation of their human players has long been on exponential growth in Artificial Intelligence. One of the most foundational and widely implemented decision-making models is the *Minimax algorithm*, first time introduced by John von Neumann in 1928.

In this paper, the Minimax algorithm has been implemented on the *game of SIM*, played on a complete graph K_6, where two players alternately color its edges with the goal of avoiding a monochromatic triangle. To showcase the performance and limits of this search, we further implement and analyze its modifications: First, we introduce *Alpha-Beta pruning* to optimize the search tree by cutting off unnecessary subtrees and therefore minimizing the number of visited game states without sacrificing the optimality of the final decision. Additionally, we implement a *Depth-limited search* approach and with the use of *heuristics* bound the search space at a specified depth D, further reducing execution time.

Finally, a detailed analysis of all these algorithms is presented, backed by runtime measurements across different search depths and graph sizes (specifically comparing K_6 with K_7 and K_{10}).

1 Game of SIM

As an important field of combinatorics, the study of positional games explores various types of games, ranging from the recreational classics such as the simple Tic-Tac-Toe, to purely abstract games played on graphs. One such example is the game of SIM, a well known pencil-and-paper game, first time introduced by Gustavus Simmons in 1969 [1].

1.1 The setting of the game

As with every positional game, there are two players who alternately take turns trying to fulfill a specific winning condition. In this game, they aim to avoid creating a clique of size 3 and instead force their opponent to do so.

Definition 1. In the game of SIM

- the board is the edge set $E(K_6)$ of the complete graph K_6;
- two players, Red and Blue, alternately claim previously unclaimed edges of K_6, each claiming exactly one egde per move;
- a player loses as soon as he claims a monochromatic K_3.

It would be only natural to presume that if neither player creates a monochromatic triangle by the end of the game, the game ends in a draw. However, with the Ramsey number being $R(3,3) = 6$, we know that any two-coloring of edges of K_6 contains a monochromatic K_3. Consequently, the game can never actually result in a draw and one player must possess a winning strategy. Exhaustive computer searches have shown that it is Blue (the second player) who can always win the game of SIM and we now give a simple example of his strategy (see [2]).

1.2 A winning strategy for the second player

Before describing the exact strategy, we will introduce a small amount of terminology.

Definition 2. In the game of SIM:

- an edge is *allowed* for a player if claiming it does not create a monochromatic K_3 in that player's graph.
- a set of edges on a graph M is *allowed* for a player if claiming it fully does not create a monochromatic K_3 in that player's graph.
- a *mini-board* is the smallest complete subgraph of the board that contains all currently claimed edges and at least one edge allowed for Blue.

Whenever it is Blue's turn, they proceed as follows:

1. Fix an arbitrary mini-board M of the current state of the game.

2. Among all edges of M that are allowed for Blue, choose an edge that belongs to the largest number of allowed sets on M for Blue.

3. If that did not determine the move uniquely, choose an edge that belongs to the greatest number of allowed sets on M for Red.

4. If still not uniquely determined, choose any of the remaining edges.

Even though this strategy does not determine every move uniquely, it has been proven that it always leads to a win for the second player.

2 Minimax algorithm in SIM

The theoretical existence of a winning strategy in the game of SIM can be practically shown using classical game-theoretic algorithms and in this paper we rely on the Minimax algorithm. Using this approach, the computer (playing as Blue) can evaluate any given game state to determine its next move.

2.1 Definiton of Minimax

In its most general form, Minimax is a decision-making algorithm that recursively searches the game tree, trying to find the optimal move for a player, assuming that the opponent is also playing optimally. The algorithm models two players with opposing goals:

- *Maximizer* - trying to maximize the final score.
- *Minimizer* - trying to minimize the final score.

We further formalize this decision-making process as a mutual recursion between two distinct functions: $getMin$ and $getMax$.

Remark 3. Let S denote the set of all possible game states, where a state $s \in S$ is defined by the current coloring of the edges of K_6 with two colors (Blue and Red).

We define the set of available moves (unclaimed edges) in state s as:

$$A(s) = \{(i,j) \mid 0 \leq i < j < 6, \text{ the edge } (i,j) \text{ is unclaimed}\}$$

Let newState(s, e, c) be the function that returns a new state obtained by coloring the edge $e \in A(s)$ with color $c \in \{\text{blue}, \text{red}\}$.

2.1.1 Function getMax

Maximizer's goal is to secure a value of 1.0 (representing a win) and, if possible, avoid the losing value of -1.0 (completing a monochromatic blue K_3).

For a non-terminal state s where it is Blue's turn, function $getMax(s)$ can be formally defined as:

$$getMax(s) = \max_{e \in A(s)} val(s, e, \text{blue}),$$

where the evaluation of a specific edge $e \in A(s)$ can be represented as:

$$val(s, e, \text{blue}) = \begin{cases} -1.0, & \text{if newState}(s, e, \text{blue}) \text{ contains a blue } K_3, \\ 1.0, & \text{if } A(s) \setminus \{e\} = \emptyset \text{ (no more edges)}, \\ getMin(\text{newState}(s, e, \text{blue})), & \text{otherwise.} \end{cases}$$

2.1.2 Function getMin

Minimizer is, on the other hand, seeking to minimize the game's outcome toward -1.0.

For a non-terminal state s where it is Red's turn, function $getMin(s)$ can be formally defined as:

$$getMin(s) = \min_{e \in A(s)} val(s, e, \text{red})$$

where the evaluation of a specific edge $e \in A(s)$ can be represented as:

$$val(s, e, \text{red}) = \begin{cases} 1.0, & \text{if newState}(s, e, \text{red}) \text{ contains a red } K_3, \\ -1.0, & \text{if } A(s) \setminus \{e\} = \emptyset \text{ (no more edges)}, \\ getMax(\text{newState}(s, e, \text{red})), & \text{otherwise.} \end{cases}$$

2.1.3 Implementation Details

In the actual implementation, the algorithm searches through all pairs (i, j) such that:

- $0 \leq i < j <$ SIZE, and
- the edge $e^* := (i, j)$ that they form is unclaimed, i.e., $e^* \in A(s)$.

The state space searching is achieved by temporarily coloring all such edges e^* using `colorTheEdge(i, j, color)` method and backtracking its state with `colorTheEdge(i, j, null)` once the recursive branch is evaluated.

The implementation of functions $getMax$ and $getMin$ is given in 2.1.3 and 2.1.3.

```
private MinMaxRes getMax() {
    MinMaxRes res = new
        MinMaxRes(-1, -1, -1);
    for(int i=0; i<SIZE; i++) {
        for(int j=i+1; j<SIZE; j++) {
            if(canPlay(i, j)) {
                colorTheEdge(i, j,
                    Color.blue);
                float val;
                if(gameEnded(i, j,
                    Color.blue)) {
                    val = -1.0f;
                } else
                    if(gameDone()) {
                    val = 1.0f;
                } else {
                    val =
                        getMin().val;
                }
                if(val > res.val) {
                    res.val = val;
                    res.v1 = i;
                        res.v2 = j;
                }
                colorTheEdge(i, j,
                    null);
            }
        }
    }
    return res;
}
```

The Maximizing Method

```
private MinMaxRes getMin() {
    MinMaxRes res = new
        MinMaxRes(-1, -1, 2);
    for(int i=0; i<SIZE; i++) {
        for(int j=i+1; j<SIZE; j++) {
            if(canPlay(i, j)) {
                colorTheEdge(i, j,
                    Color.red);
                float val;
                if(gameEnded(i, j,
                    Color.red)) {
                    val = 1.0f;
                } else
                    if(gameDone()) {
                    val = -1.0f;
                } else {
                    val =
                        getMax().val;
                }
                if(val < res.val) {
                    res.val = val;
                    res.v1 = i;
                        res.v2 = j;
                }
                colorTheEdge(i, j,
                    null);
            }
        }
    }
    return res;
}
```

The Minimizing Method

2.2 Properties of the Algorithm

While Minimax in theory always finds the best move for a player (and therefore is *complete and optimal*), it is not always possible to complete the search of the whole state space. This can happen if the game is too complex and has too many possible states, making the search process too time consuming.

2.2.1 Time complexity

Fundamentally, Minimax is structured as a *Depth-first search (DFS)* algorithm, which means its time complexity is expressed as

$$O(b^d),$$

where b is the branching factor and d is the maximum depth of the tree. [3]

However, the condition `canPlay(i, j)` filters out the edges that have already been claimed. Therefore, the branching factor b decreases by one at each level of the tree.

Since our game is played on a complete graph K_6, we know there are $\binom{6}{2} = 15$ edges that can be claimed in the beginning and, as the computer player goes second and in its first move has 14 available edges, the total number of nodes generated in the worst possible case is

$$14 + 14 \cdot 13 + 14 \cdot 13 \cdot 12... + 14! = \sum_{i=1}^{14} \frac{14!}{(14-i)!}$$

.

Consequently, the time complexity for our search can therefore be described as factorial ($O(b!)$) and scale as $O(14!)$, where $14! \approx 87 \times 10^9$.

2.2.2 Space complexity

The space complexity of a full-search Minimax algorithm is, on the other hand, fairly efficient. As it is searching the game tree using DFS, it only needs to store the active path from the root node to the current node, giving us $O(d \cdot c)$ space complexity, where d is the maximum depth of the tree and c is the space allocated per individual recursive call.

In the game of SIM, the maximum depth can never be larger than the maximum number of available edges, $d = 14$. Further, the algorithm modifies the same, shared graph in every recursion call using the `colorTheEdge` method, instead of allocating a new one. The only structures that actually get instantiated in memory during each call are the `MinMaxRes` objects, but their allocation falls under a constant factor which brings us to the space complexity of $O(d \cdot 1) = O(d)$, which for K_6 scales as $O(14)$.

3 Alpha-Beta pruning

Due to the large state space of about 87 billion possible states of the game, brute-force approach that searches the whole game tree becomes unusable in real-time as it takes too long on the very first turn. To overcome this limitation, we use Alpha-Beta pruning to cut down unnecessary branches of our game tree.

3.1 Definition and implementation details

The core principle of Alpha-Beta pruning is to maintain these two values throughout the whole search:

- α, the maximum score the Maximizer is guaranteed to achieve up to that point, and
- β, the minimum score the Minimizer is guaranteed to achieve.

As we search the tree, if at any point we encounter a state where $\alpha \geq \beta$, it means that the opponent would never allow this path to be reached during optimal play. Thus, the search does not continue to any further sibling nodes at this level, effectively *pruning* the subtree, and just returns the current best/worst evaluation.

In our implementation, this optimization is achieved by passing the α and β variables across the recursive calls, updating them accordingly and cutting the branches when possible:

```
if (val > alpha) {
    alpha = val;
}
if (alpha >= beta) {
    return res;
}
```

Maximizer (`getMax`)

```
if (val < beta) {
    beta = val;
}
if (alpha >= beta) {
    return res;
}
```

Minimizer (`getMin`)

3.2 Time complexity

When analyzing the time complexity of Alpha-Beta pruning [4], the performance is dictated by the order in which the branches are evaluated. In the worst-case scenario (where the moves are searched in the worst possible order) the algorithm achieves no pruning at all, having a time complexity identical to a full search, $O(b!)$.

However, in the best-case scenario, the algorithm can achieve optimal pruning. This occurs when the best available move is always evaluated first at every node:

- for the Maximizer, the branch that brings the highest evaluation, and
- for the Minimizer, the branch that brings the lowest,

and if this condition is met, the search can immediately be cut off (because $\alpha \geq \beta$).

In a standard game tree where the branching factor b is constant, this optimization alternates between fully expanding a node and immediately cutting off the remaining branches on the next level. This creates a branching pattern of $b \to 1 \to b \to 1 \to \ldots$ down to the depth d. Summing these nodes across the levels, we get the following sum:

$$b + b + b^2 + b^2 + \ldots + b^{d/2} + b^{d/2}$$

yielding the time complexity of

$$O(b^{d/2})$$

.

However, in the game of SIM, the branching factor is not constant and decreases by 1 at each level. When we apply the same cutoff pattern, we get that the number of nodes at the final level equals to:

$$b \cdot 1 \cdot (b-2) \cdot 1 \cdot (b-4) \cdot 1 \cdot \ldots = b!!$$

Therefore, time complexity for the optimal Alpha-Beta pruning of the game of SIM mirrors the standard improvement $O(b^{d/2})$ with the time complexity of $O(b!!)$.

Remark 4. To achieve this scenario, we can generate a list of possible moves and evaluate them, prioritizing those with the highest probability of causing a search cut-off. This is done using a heuristic evaluation function, which estimates the strength of each move based on the distribution of threats on the board (its implementation is discussed in Section 4.1). The implementation of this Optimal pruning on the game of SIM can be found in `class GameLogicImpossibleOptimal`.

4 Depth-limited search using Heuristic evaluation

Even though faster, previous cuts still carry one major limitation: they are forced to search all the way to the very end of the game tree. To make the decision-making even faster and more flexible, we can introduce a search limit.

Instead of letting the recursion reach the terminal leaves, we can force the algorithm to stop searching once it reaches a certain depth. We can then replace the absolute win/loss scores (1 and -1) with an estimation of how "good" the current state looks for each player.

4.1 Heuristic evaluation function

A heuristic is an evaluation function that estimates how good a game state is without searching all the way to the bottom of the tree. The more accurate the heuristic function, the better the decision-making of our bot.

In the classes that implement depth-limited search (`GameLogicEasy` and `GameLogicMedium`), this value is calculated using the `evaluate()` method:

```
private float evaluate() {
    int redThreats = threats.get(Color.red);
    int blueThreats = threats.get(Color.blue);
    return (float)(redThreats - blueThreats) / NUM_OF_TRIANGLES;
}
```

We can break down its components:

- *The concept of threats:* In the game of SIM, a "threat" represents an open angle: a structure consisting of two neighboring edges of the same color, with the third closing edge still unclaimed.
- *Threat difference:* If a player is forced to color that third, unclaimed edge on their turn, they will form a monochromatic triangle and immediately lose. Therefore, a state is favorable for us if we have fewer threats while our opponent has many. Then the quality of the current state for the computer can be calculated simply as the difference between its opponent's threats and its own threats (`redThreats - blueThreats`).
- *Normalization to the* $[-1, 1]$ *range:* Since our game-over values are set to 1 and -1, we need to scale our heuristic evaluation to fit within the same range. We achieve this by dividing the threat difference by `NUM_OF_TRIANGLES` (which

is $\binom{6}{3} = 20$ in a K_6 graph). Because the number of monochromatic open angles on the board can never reach the total number of possible triangles, this division guarantees that the value of the ratio remains strictly less than 1.

4.2 Properties of the algorithm

We can analyze both the time and space complexity of this depth-limited approach:

- **Time complexity:** In a standard depth-limited DFS, the worst-case time complexity is typically expressed as $O(b^D)$, where b is the branching factor and D is the maximum depth limit. As, in the game of SIM, the number of available edges decreases by one at each level of the game tree, the total number of nodes visited comes to:

$$d + d(d-1) + d(d-1)(d-2) + \cdots + \frac{d!}{(d-D)!},$$

  which brings us to the worst-case time complexity of:

$$O\left(\frac{d!}{(d-D)!}\right)$$

  .

- **Space complexity:** By this limitation of the search, the space complexity is reduced from $O(d)$ for a full-tree search down to:

$$O(D), \text{ where } D < d.$$

  In our implementation, this limit is set to $D = 2$ in the `GameLogicMedium` class, and $D = 1$ in the `GameLogicEasy` class.

5 Comparison analysis

To provide a full time and space comparison of the previously described algorithms, we tested them across different values of the variables b, d, and D, and measured the time it took for the computer to play its first move with each runtime represented as the average of 10 independent runs.

5.1 Time and space complexity comparison

Table 1 illustrates how different searches and algorithms perform under a fixed graph size (K_6) with varying depth limits.
Key insights:

- *The impracticability of full Minimax search:* Without any pruning or depth limits, the game tree for SIM is simply too huge and practically impossible to search through in real-time.

Algorithm	Time Compl.	Space Compl.	Visited Nodes	First Move Time
Full Search (Minimax)	$O(b!)$	$O(d)$	$\approx 87 \times 10^9$	/
Standard Alpha-Beta	$O(b!)$	$O(d)$	Higher	529.54 ms
Optimal Alpha-Beta	$O(b!!)$	$O(d)$	Lower	636.01 ms
Depth-Limited ($D = 6$)	$O\left(\frac{d!}{(d-D)!}\right)$	$O(D)$	$< 2.43 \times 10^6$	30.13 ms
Depth-Limited ($D = 3$)	$O\left(\frac{d!}{(d-D)!}\right)$	$O(D)$	$< 2,381$	3.45 ms
Depth-Limited ($D = 1$)	$O(d)$	$O(D)$	15	1.17 ms

Table 1: Performance comparison of different search algorithms for the first move ($d = 14$).

- *The not so optimal ordering:* Even though the Optimal Alpha-Beta pruning algorithm is theoretically better because it visits fewer nodes, in our implementation, it is actually about 106 ms slower than the Standard Alpha-Beta. This showed that sorting the moves, creating new objects, and constantly evaluating states takes more time than what we actually save by pruning the extra branches, meaning that a theoretically optimal algorithm is only beneficial if the cost of its heuristic evaluation is significantly lower than the cost of traversing the pruned edges.
- *The efficiency of depth limits:* At depth $D = 6$, the computer makes a decision in what is more than 20 times faster than the pruned full search, and its play still feels very smart and correct. This speedup is better highlighted in Figure 1.

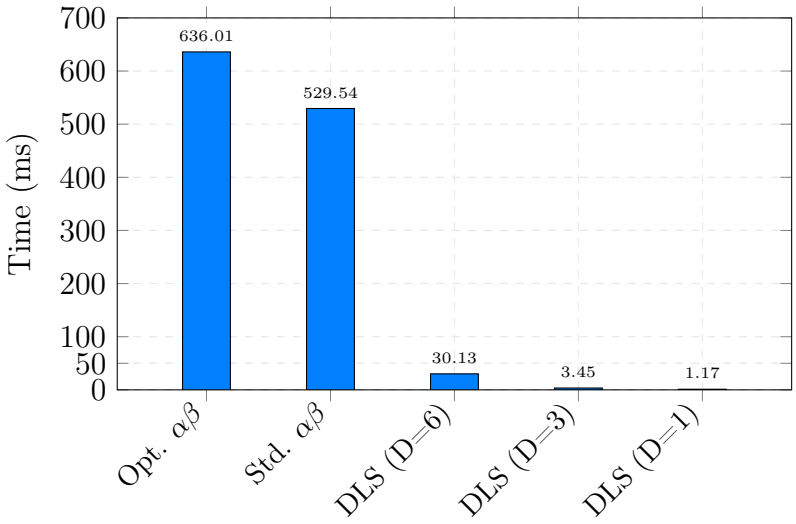


Figure 1: First move execution time by algorithm

5.2 Comparison across different graph sizes

Table 2 illustrates how long it took the algorithms to calculate their first move depending on the size of the graph on which it was implemented, specifically for K_6, K_7 (consisting of 21 edges) and K_{10} (consisting of 45 edges).

Algorithm / Size	K_6 $(d = 14)$	K_7 $(d = 20)$	K_{10} $(d = 44)$
Alpha-Beta pruning	529.54 ms	/	/
Depth-limited search $(D = 6)$	30.13 ms	113.90 ms	974.11 ms
Depth-limited search $(D = 3)$	3.45 ms	6.06 ms	9.81 ms
Depth-limited search $(D = 1)$	1.17 ms	1.36 ms	1.97 ms

Table 2: Algorithm performances across different graph sizes (K_6, K_7, and K_{10}).

Key insights:

- *The impracticability of full Alpha-Beta pruning:* More available edges makes a pruned, full-depth search impossible for real-time play. This shows how, for even slightly larger graphs, depth-limiting is no longer just an option, but the only way to make the game playable.
- *The efficiency of depth limits:* At $D = 6$, the bot is still able to make decisions in under a second, even on K_{10}.

Conclusion

In this paper, we have provided a thorough analysis of the Minimax algorithm applied to the game of SIM, all through the examples of different algorithmic modifications of its full search. We have shown how the transition from the full game tree to optimized pruning significantly impacts computational performance, while introducing heuristic evaluations and depth limits allows the computer to make almost perfect moves using real-time decisions.

All of this is supported by a practical implementation of different gameplay difficulty levels, where each level corresponds to a specific search algorithm:

- `class GameLogicFullSearch` - Full tree search,
- `class GameLogicImpossible` - Alpha-Beta pruning,
- `class GameLogicImpossibleOptimal` - Optimal Alpha-Beta pruning,
- `class GameLogicMedium` - Depth-limited search with $D = 2$,
- `class GameLogicEasy` - Depth-limited search with $D = 1$.

References

[1] Gustavus J. Simmons. *The game of SIM.* Journal of Recreational Mathematics vol. 2, 1969.

[2] Weronika Wrzos-Kaminska. *A simpler winning strategy for Sim.* arXiv, 2020.

[3] Przemysław Klesk. *The Minimax Algorithm.* West Pomeranian University of Technology in Szczecin.

[4] Don Ma. *Optimization of Alpha-Beta pruning based on heuristic algorithm*, volume 6. Applied and Computational Engineering, 06 2023.